

# 04 Data Preprocessing

## Markdown Cell 1

Experiment 4 — Data Preprocessing

Source: Data Preprocessing PDF

The notebook follows the preprocessing workflow shown in the PDF: missing-value imputation, label encoding, one-hot encoding, train/test split, and standardization.

## Code Cell 2

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Import dataset
data_preprocessing = pd.read_csv("data_preprocessing.csv")

# Display dataset
data_preprocessing.head()
```

## Code Cell 3

```
# Extracting Independent Variable
X = data_preprocessing.iloc[:, :-1].values

# Extracting Dependent Variable
y = data_preprocessing.iloc[:, -1].values

print("X:")
print(X)

print("\ny:")
print(y)
```

## Code Cell 4

```
from sklearn.impute import SimpleImputer

# Replacing missing values with mean
imputer = SimpleImputer(
    missing_values=np.nan,
    strategy='mean'
)

# Fitting imputer
imputer.fit(X[:, 1:3])

# Replacing missing values
X[:, 1:3] = imputer.transform(X[:, 1:3])
print(X)
```

## Code Cell 5

```
# For country variable
from sklearn.preprocessing import LabelEncoder

label_encoder_X = LabelEncoder()
X[:, 0] = label_encoder_X.fit_transform(X[:, 0])
print(X)
```

## Code Cell 6

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

onehot_encoder = ColumnTransformer(
    transformers=[
        ('country', OneHotEncoder(sparse_output=False), [0])
    ],
    remainder='passthrough'
)

X = onehot_encoder.fit_transform(X)
print(X)
```

## Code Cell 7

```
label_encoder_y = LabelEncoder()
y = label_encoder_y.fit_transform(y)
print(y)
```

## Code Cell 8

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=0
)

print("X_train:")
print(X_train)
print("\nX_test:")
print(X_test)
print("\ny_train:")
print(y_train)
print("\ny_test:")
print(y_test)
```

## Code Cell 9

```
from sklearn.preprocessing import StandardScaler

st_x = StandardScaler()
X_train = st_x.fit_transform(X_train)
X_test = st_x.transform(X_test)

print("X_train:")
print(X_train)
print("\nX_test:")
print(X_test)
```